PyTorch 입문

텐서 · 자동 미분 · `nn` · `optim` · `utils.data` · MNIST 분류 코드 읽기

딥러닝개론및연습 · 2026년 2학기

이번 시간의 핵심 질문

이번 시간의 핵심 질문

- 지금까지 우리는 NumPy로 **모든 것을 직접** 만들었다: 계층의 순전파와 역전파, 기울기 갱신, 미니배치 나누기
- 새로운 계층을 쓸 때마다 역전파 식을 **손으로 유도**해야 할까?
- 수십만 장의 이미지와 수백만 개의 매개변수를 **GPU**로 빠르게 계산하려면?
- 실제 연구와 산업에서는 신경망을 어떤 도구로 만들까?

$\Rightarrow$ **딥러닝 프레임워크: PyTorch**

학습목표

1. PyTorch가 무엇이고, 무엇을 대신해 주는지 설명할 수 있다
2. 텐서(tensor)를 만들고 shape, dtype, device를 다룰 수 있다
3. 자동 미분(autograd)이 우리가 배운 역전파와 같은 일을 한다는 것을 확인한다
4. `torch.nn`으로 모델을, `torch.optim`으로 갱신 규칙을 만든다
5. `torch.utils.data`의 `Dataset`, `DataLoader`로 미니배치를 만든다
6. 학습 루프의 5줄(순전파 $\rightarrow$ 손실 $\rightarrow$ 기울기 초기화 $\rightarrow$ 역전파 $\rightarrow$ 갱신)을 이해한다
7. MNIST 분류 코드 `mnist_torch.py`를 처음부터 끝까지 읽고 실행한다

목차

PyTorch란 무엇인가

텐서 (Tensor)

자동 미분 (Autograd)

`torch.nn`: 계층과 모델

`torch.optim`: 매개변수 갱신

`torch.utils.data`: 데이터와 미니배치

학습 루프의 5줄

`mnist_torch.py` 코드 읽기

정리와 실습

1

PyTorch란 무엇인가

지금까지 우리가 직접 만든 것

할 일	NumPy로 직접 만든 것	어려운 점
데이터 저장, 계산	`np.array`, `X @ W + B`	CPU에서만 계산
계층	`Affine`, `Relu`, `Convolution`, ⋯ 의 `forward`	계층마다 클래스 작성
역전파	각 계층의 `backward`	식을 **손으로 유도**, 틀리기 쉬움 (기울기 검사)
매개변수 갱신	`SGD`, `Momentum`, `Adam` 클래스	매개변수 딕셔너리 관리
미니배치	`np.random.permutation`으로 섞고 자르기	매번 같은 코드 반복

- 원리를 이해하려면 직접 만드는 것이 가장 좋다. **그래서 지금까지 직접 만들었다.**
- 하지만 큰 모델을 실제로 학습할 때 이것을 매번 직접 짤 수는 없다.

딥러닝 프레임워크

- **딥러닝 프레임워크**: 위의 일들을 미리 만들어 둔 라이브러리
  - **PyTorch** (2016, Meta AI에서 시작 → 현재 Linux 재단 산하 PyTorch Foundation)
  - TensorFlow / Keras (2015, Google), JAX (Google)
- 현재 연구 논문과 공개 모델(예: Hugging Face의 언어 모델들)의 대부분이 **PyTorch**로 작성된다
- Python 코드처럼 **한 줄씩 실행**되므로 NumPy를 아는 사람이 배우기 쉽다

PyTorch를 한 문장으로

GPU에서 동작하고, 기울기를 자동으로 계산하는 NumPy + 신경망 부품 모음

PyTorch의 구성: 이번 시간에 배울 다섯 가지

모듈	역할	우리 코드에서 대신하는 것
`torch`	**텐서**: GPU에서도 계산되는 다차원 배열	`np.array`, `np.exp`, ⋯
`torch.autograd`	**자동 미분**: `loss.backward()` 한 줄로 모든 기울기 계산	모든 `backward`
`torch.nn`	계층, 모델, 손실 함수	`Affine`, `Relu`, `SoftmaxWithCrossEntropy`
`torch.optim`	매개변수 갱신 규칙	`optimizers.py`
`torch.utils.data`	데이터 묶음, 미니배치	`permutation`으로 섞고 자르기

- `torch`가 라이브러리 전체이고, `nn`, `optim`, `utils.data`는 그 안의 **하위 모듈**이다
- NumPy에 `numpy.linalg`, `numpy.random`이 있는 것과 같다. 따로 설치할 필요가 없다

설치와 확인

- 설치 (CPU 버전): `pip install torch`
- GPU(NVIDIA CUDA)용은 `pytorch.org`에서 자기 환경에 맞는 설치 명령을 골라 쓴다
- Google Colab에는 이미 설치되어 있고, GPU도 무료로 쓸 수 있다

```
import torch
print("torch version:", torch.__version__)
print("GPU available:", torch.cuda.is_available())
```

```
torch version: 2.14.1+cpu
GPU available: False
```

- 이 강의의 모든 결과는 GPU 없이 **CPU**로 실행한 것이다. MNIST 정도는 CPU로 충분하다
- 예제 코드: `torch_basics.py` (이번 시간의 작은 예제 모음), `mnist_torch.py`

2

텐서 (Tensor)

텐서 만들기

- **텐서**: PyTorch의 다차원 배열. NumPy의 `ndarray`와 거의 같다

```
a = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])
print(a)
print(a.shape, a.dtype, a.device)

print(torch.zeros(2, 3))
print(torch.arange(6).reshape(2, 3))
print(torch.randn(2, 3))   # N(0, 1)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
torch.Size([2, 3]) torch.float32 cpu
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[0, 1, 2],
        [3, 4, 5]])
tensor([[ 1.5410, -0.2934, -2.1788],
        [ 0.5684, -1.0845, -1.3986]])
```

- 모든 텐서는 세 가지 속성을 가진다: **`shape`** (모양), **`dtype`** (자료형), **`device`** (CPU / GPU)
- `torch.zeros(2, 3)`: NumPy의 `np.zeros((2, 3))`와 달리 괄호를 하나 덜 써도 된다

자료형 (dtype)

dtype	쓰는 곳	만들어지는 경우
`torch.float32` (`float`)	입력 데이터, 가중치	`torch.tensor([1., 2.])` (기본값)
`torch.int64` (`long`)	**클래스 번호** (정답 레이블)	`torch.tensor([3, 7])`
`torch.float64` (`double`)	거의 쓰지 않는다	NumPy 배열을 그대로 변환할 때

- NumPy의 기본은 `float64`, PyTorch의 기본은 `float32`: 메모리가 절반이고 GPU에서 훨씬 빠르다. 정밀도는 신경망 학습에 충분하다
- 자료형이 섞이면 **오류**가 난다. 초보자가 가장 많이 만나는 오류 두 가지:

자주 보는 오류

`mat1 and mat2 must have the same dtype, but got Double and Float`
→ `float64` 입력을 `float32` 가중치에 넣었다. `dtype=torch.float32`로 변환

`expected target dtype to be Long or Byte, but got Float`
→ 손실 함수의 정답 레이블은 `int64`여야 한다

NumPy와 오가기

```
x_np = np.array([1.0, 2.0, 3.0])   # float64
x_t = torch.tensor(x_np)
print(x_t, x_t.dtype)
print(torch.tensor(x_np, dtype=torch.float32))
print(a.numpy())
```

```
tensor([1., 2., 3.], dtype=torch.float64)
  torch.float64
tensor([1., 2., 3.])
[[1. 2. 3.]
 [4. 5. 6.]]
```

- `torch.tensor(x_np)`: NumPy의 `float64`를 **그대로** 가져온다 → `dtype=torch.float32`를 붙인다
- `tensor.numpy()`: 다시 NumPy 배열로 (그림 그리기, 결과 저장 등)
  - GPU에 있는 텐서는 먼저 `.cpu()`로 가져와야 한다
  - 기울기를 추적 중인 텐서는 `.detach().numpy()`
- 원소 하나짜리 텐서 → Python 숫자: `.item()` (예: `loss.item()`)

연산: NumPy와 거의 같다

```
W = torch.ones(3, 4)
print(a @ W)                # (2,3)@(3,4)
b = torch.tensor([1., 2., 3.])
print(a + 10 * b)           # broadcasting
print(a.sum(dim=0))
print(a.sum(dim=1))
print(a.argmax(dim=1))
print(a[1, 2], a[1, 2].item())
```

```
tensor([[ 6.,  6.,  6.,  6.],
        [15., 15., 15., 15.]])
tensor([[11., 22., 33.],
        [14., 25., 36.]])
tensor([5., 7., 9.])
tensor([ 6., 15.])
tensor([2, 2])
tensor(6.) 6.0
```

- 행렬곱 `@`, 원소별 연산, **브로드캐스팅**, 인덱싱, `.T`, `reshape`: NumPy와 같다
- 차이: NumPy의 `axis` 대신 **`dim`**을 쓴다 (`axis`도 받아 주지만 `dim`이 표준)

NumPy ↔ PyTorch 대응표

NumPy	PyTorch
`np.array(L)`	`torch.tensor(L)`
`np.zeros((2, 3))`, `np.ones((2, 3))`	`torch.zeros(2, 3)`, `torch.ones(2, 3)`
`np.random.randn(2, 3)`	`torch.randn(2, 3)`
`np.random.seed(0)`	`torch.manual_seed(0)`
`x.sum(axis=0, keepdims=True)`	`x.sum(dim=0, keepdim=True)`
`np.argmax(x, axis=1)`	`x.argmax(dim=1)`
`np.exp(x)`, `np.log(x)`, `np.maximum(0, x)`	`torch.exp(x)`, `torch.log(x)`, `torch.relu(x)`
`np.concatenate([x, y])`	`torch.cat([x, y])`
`x.astype(np.float32)`	`x.float()` 또는 `x.to(torch.float32)`
`x.size` (원소 수)	`x.numel()`
`x.shape`, `x.T`, `x @ W`, `x.reshape(3, 2)`	같음

device: CPU와 GPU

- 텐서는 **CPU 메모리** 또는 **GPU 메모리**에 있다 (`a.device`)
- `.to(device)`로 옮긴다. GPU에 있는 텐서끼리의 연산은 GPU에서 병렬로 실행된다

```
device = "cuda" if torch.cuda.is_available() else "cpu"   # GPU if we have one
x = x.to(device)                # data to the device
model = model.to(device)        # all parameters of the model to the device
```

규칙

함께 계산하는 텐서는 모두 같은 device에 있어야 한다. 모델을 GPU로 보냈으면 입력 미니배치도 GPU로 보낸다.

- 이렇게 써 두면 **같은 코드**가 GPU가 있으면 GPU로, 없으면 CPU로 돈다
- GPU가 빠른 이유: 행렬곱의 곱셈과 덧셈이 서로 독립이라 동시에 계산할 수 있다

3

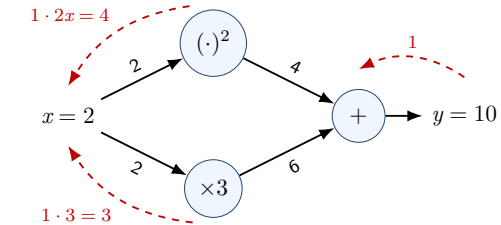
자동 미분 (Autograd)

핵심 아이디어: 순전파가 계산 그래프를 기록한다

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x       # forward
y.backward()             # backward
print(y.item(), x.grad.item())
```

```
10.0 7.0
```

- `requires_grad=True`: "이 텐서에 대한 미분이 필요하다"
- 순전파를 하는 동안 PyTorch가 **계산 그래프**를 기록
- `y.backward()`: 그래프를 거꾸로 따라가며 **연쇄법칙** 적용
- 결과는 `x.grad`에 저장: $\frac{dy}{dx} = 2x + 3 = 7$



검은 화살표: 순전파 (값), 빨간 점선: 역전파 (기울기)

우리가 유도한 역전파 식과 비교

- Affine 계층 $A = XW + B$, 손실 $L = \frac{1}{2}\sum(A - T)^2$. 우리가 유도한 식: $\frac{\partial L}{\partial W} = X^{\mathsf{T}}\frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial B} = \sum_{\text{행}} \frac{\partial L}{\partial A}$

```
X = torch.tensor([[1., 2.], [3., 4.]])
W = torch.tensor([[0.5], [-1.0]], requires_grad=True)
B = torch.tensor([[0.1]], requires_grad=True)
T = torch.tensor([[1.], [0.]])
A = X @ W + B
L = 0.5 * ((A - T) ** 2).sum()
L.backward()                                    # autograd
dA = (A - T).detach()                           # dL/dA, by hand
print(W.grad.T, (X.T @ dA).T)
print(B.grad, dA.sum(dim=0, keepdim=True))
```

```
tensor([[ -9.6000, -14.4000]]) tensor([[ -9.6000, -14.4000]])
tensor([[-4.8000]]) tensor([[-4.8000]])
```

- 정확히 같다. **우리가 계층마다 손으로 짠 `backward`를 autograd가 대신 해 준다**

주의 1: 기울기는 더해진다 → 매번 0으로

```
x = torch.tensor(2.0, requires_grad=True)
for k in range(3):
    (x ** 2).backward()       # d/dx = 2x = 4
    print(x.grad.item())
x.grad.zero_()
print(x.grad.item())
```

```
4.0
8.0
12.0
0.0
```

- `backward()`는 기울기를 `.grad`에 **덮어쓰지 않고 더한다** (한 변수가 여러 경로로 쓰이면 모든 경로의 기울기를 더해야 하기 때문)
- 따라서 미니배치마다 `backward()` **전에** 기울기를 0으로: 학습 코드의 `optimizer.zero_grad()`

잊으면?

이전 미니배치들의 기울기가 계속 쌓여서 엉뚱한 방향으로 갱신된다. 오류 메시지도 나지 않으므로 찾기 어렵다.

주의 2: 기울기가 필요 없을 때는 torch.no_grad()

```
with torch.no_grad():          # no graph is recorded inside
    z = x * 3
print(z.requires_grad)         # False
```

- 평가(정확도 계산)나 예측에는 역전파가 필요 없다
  - 그래프를 기록하지 않으므로 **메모리가 적게 들고 빠르다**
  - 함수 전체에 적용하려면 데코레이터 `@torch.no_grad()`를 함수 위에 붙인다 (evaluate에서 자세히)
- `.detach()`: 그래프에서 떼어 낸 텐서 (값은 같고 기울기 추적만 끊음)
- `nn`의 계층이 가진 가중치는 처음부터 `requires_grad=True`이다. 우리가 따로 지정할 일은 거의 없다

정리: 자동 미분

순전파 계산을 그냥 쓰면 → 그래프가 기록되고 → `loss.backward()`가 모든 매개변수의 `.grad`를 채운다.

4

torch.nn: 계층과 모델

nn.Module: 모든 계층과 모델의 틀

- PyTorch의 계층과 모델은 모두 nn.Module을 **상속**한 클래스이다
- 우리가 만든 계층 클래스와 비교:

우리 NumPy 계층	**nn.Module**
__init__: 가중치 받기	__init__: 가중치를 **스스로 만들고 초기화**
forward(X)	forward(x). 호출은 layer(x)처럼 함수처럼 한다
backward(dout)	**없음**. autograd가 자동으로
self.params 딕셔너리	.parameters(): 모든 매개변수를 자동으로 모아줌
train_flg 인자 (Dropout)	model.train() / model.eval()

- 모델 안에 계층을 넣으면, 계층의 매개변수도 모델의 매개변수로 **자동 등록**된다
- model.to(device) 한 줄로 모든 매개변수가 GPU로 간다

nn.Linear: Affine 계층

```
fc = nn.Linear(784, 100)
print(fc)
print(fc.weight.shape, fc.bias.shape)
xb = torch.randn(5, 784)      # mini-batch of 5
print(fc(xb).shape)
print(torch.allclose(fc(xb),
          xb @ fc.weight.T + fc.bias))
```

```
Linear(in_features=784,
       out_features=100, bias=True)
torch.Size([100, 784]) torch.Size([100])
torch.Size([5, 100])
True
```

- 우리 Affine 계층과 같은 계산이다. 단, **가중치의 모양이 반대**:

  우리: $A = XW + B, \quad W \in \mathbb{R}^{784\times100}$ `nn.Linear`: $A = XW^{\mathsf{T}} + \boldsymbol{b}, \quad W \in \mathbb{R}^{100\times784}$

- 가중치는 작은 난수로 **자동 초기화**된다 (입력 개수에 맞춘 범위의 균등분포)
- 미니배치 (`N, 784`)가 들어가면 (`N, 100`)이 나온다. 첫 번째 차원은 항상 **데이터 개수**

활성화 함수와 nn.Sequential

```
relu = nn.ReLU()
print(relu(torch.tensor([-2., 0., 3.])))

model = nn.Sequential(
    nn.Linear(784, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
)
for name, p in model.named_parameters():
    print(name, tuple(p.shape))
print(sum(p.numel() for p in model.parameters()))
```

```
tensor([0., 0., 3.])
0.weight (100, 784)
0.bias (100,)
2.weight (10, 100)
2.bias (10,)
79510
```

매개변수 수:

$$\underbrace{784 \cdot 100 + 100}_{\text{첫 Linear}} + \underbrace{100 \cdot 10 + 10}_{\text{둘째 Linear}} = 79510$$

- `nn.Sequential`: 계층들을 **순서대로** 통과시키는 모델. 우리 코드의 `self.layers` 리스트와 같다
- 이름 0, 2는 Sequential 안의 순번. 1번 ReLU는 매개변수가 없다

주요 계층: 우리가 만든 계층과의 대응

우리 NumPy 계층	`torch.nn`	배운 시간
`Affine(W, B)`	`nn.Linear(in, out)`	다중분류
`Relu()`, `Sigmoid()`	`nn.ReLU()`, `nn.Sigmoid()`	다중분류
`SoftmaxWithCrossEntropy()`	`nn.CrossEntropyLoss()`	다중분류
`SGD`, `Momentum`, `Adam`	`optim.SGD`, `optim.Adam`	학습 기술
`Dropout(0.5)`	`nn.Dropout(0.5)`	학습 기술
`BatchNormalization`	`nn.BatchNorm1d` / `2d`	학습 기술
`Convolution(W, B, stride, pad)`	`nn.Conv2d(C_in, C_out, k, stride, pad)`	CNN
`Pooling(2, 2)`	`nn.MaxPool2d(2, 2)`	CNN
`Flatten()`	`nn.Flatten()`	CNN

- 이름과 인자만 조금 다를 뿐, 지금까지 배운 계층이 **모두** 들어 있다
- 차이는 하나: **`backward`를 우리가 쓰지 않는다**

nn.CrossEntropyLoss: softmax + cross entropy

```
loss_fn = nn.CrossEntropyLoss()
U = torch.tensor([[2.0, 1.0, 0.1],      # scores
                  [0.5, 2.5, 0.3]])
t = torch.tensor([0, 1])                # labels
Y = torch.softmax(U, dim=1)
manual = -torch.log(Y[torch.arange(2), t]).mean()
print(Y)
print(loss_fn(U, t).item(), manual.item())
```

```
tensor([[0.6590, 0.2424, 0.0986],
        [0.1086, 0.8025, 0.0889]])
0.3185397 0.3185397
```

세 가지를 기억하자

1. 입력은 softmax **전의 점수** U (logits). softmax는 손실 함수 안에서 계산된다
   $\rightarrow$ 모델의 마지막에 softmax를 **넣지 않는다**. 확률이 필요할 때만 따로 `torch.softmax`
2. 정답은 one-hot이 아니라 **클래스 번호** (int64): `[0, 1]`
3. 미니배치에 대한 **평균**을 돌려준다 (우리 코드에서 h로 나눈 것과 같다)

model.train() / model.eval(): 모드 스위치

- 모든 nn.Module은 self.training(True/False) 값을 가진다. 만든 직후에는 True
- model.train()은 **학습을 시키는 함수가 아니다**: 모델과 그 안의 **모든 계층**의 training을 True로 바꿀 뿐이다. model.eval()은 False로

```
net = nn.Sequential(nn.Linear(4, 4), nn.ReLU(),
                    nn.Dropout(0.5))
print(net.training, net[2].training)
net.eval();  print(net.training, net[2].training)
net.train(); print(net.training, net[2].training)

drop = nn.Dropout(0.5)
x = torch.ones(1, 6)
drop.eval();  print(drop(x))
drop.train(); print(drop(x))
```

```
True True
False False
True True
tensor([[1., 1., 1., 1., 1., 1.]])
tensor([[0., 2., 0., 0., 0., 2.]])
```

- 평가 모드: 그대로 통과
- 학습 모드: 무작위로 0, 살아남은 값은 $\times \frac{1}{1-0.5} = 2$

- 모드에 따라 계산이 달라지는 계층: **Dropout**, **BatchNorm** (NumPy 코드의 train_flg). 이번 MLP에는 둘 다 없지만 습관처럼 써 둔다

모드와 기울기 기록은 별개

```
lin = nn.Linear(2, 1)
lin.eval()
print(lin(torch.ones(1, 2)).requires_grad)
with torch.no_grad():
    print(lin(torch.ones(1, 2)).requires_grad)
```

```
True
False
```

	바꾸는 것	언제
`model.train()` / `model.eval()`	Dropout, BatchNorm의 **계산 방식**	학습 직전 / 평가 직전
`torch.no_grad()`	계산 그래프를 **기록할지**	평가, 예측 (역전파가 필요 없을 때)

- `model.eval()`만으로는 기울기 기록이 꺼지지 않는다 → 평가할 때는 **둘 다** 쓴다
- 학습 코드에서는 `evaluate`가 매 epoch `model.eval()`로 바꿔 두므로, `train_one_epoch`의 첫 줄에서 **다시** `model.train()`으로 돌려놓는다

그 밖에 알아 둘 것

- **`model.state_dict()`**: 모든 매개변수를 이름과 함께 담은 딕셔너리
  - `torch.save(model.state_dict(), "mnist_mlp.pt")`로 저장, `load_state_dict`로 불러오기
- **`torch.nn.functional`** (보통 `F`): 매개변수가 없는 연산의 함수 버전
  - `F.relu(x)`, `F.cross_entropy(U, t)`, `F.softmax(U, dim=1)` 등
  - 계층 객체 `nn.ReLU()`와 같은 계산. 어느 쪽을 써도 된다

5

torch.optim: 매개변수 갱신

torch.optim

```
w = torch.tensor([1.0, -2.0], requires_grad=True)
opt = torch.optim.SGD([w], lr=0.1)
loss = (w ** 2).sum()       # grad = 2w
opt.zero_grad()
loss.backward()
print(w.grad)
opt.step()                  # w <- w - lr * grad
print(w.detach())
```

```
tensor([ 2., -4.])
tensor([ 0.8000, -1.6000])
```

$$\begin{pmatrix}1\\-2\end{pmatrix} - 0.1\begin{pmatrix}2\\-4\end{pmatrix} = \begin{pmatrix}0.8\\-1.6\end{pmatrix}$$

- 만들 때 **갱신할 매개변수 목록**을 준다: 보통 `model.parameters()`
- `opt.zero_grad()`: 모든 매개변수의 `.grad`를 0으로
- `opt.step()`: 각 매개변수의 `.grad`를 써서 한 번 갱신

학습 관련 기술 시간에 배운 기법들

기법	PyTorch
SGD	`torch.optim.SGD(model.parameters(), lr=0.01)`
Momentum	`torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`
AdaGrad	`torch.optim.Adagrad(model.parameters(), lr=0.01)`
RMSProp	`torch.optim.RMSprop(model.parameters(), lr=0.001)`
Adam	`torch.optim.Adam(model.parameters(), lr=0.001)`

- 기법을 바꾸려면 **이 한 줄만** 바꾸면 된다. `zero_grad()`, `step()`의 사용법은 모두 같다
- 기법마다 필요한 상태(모멘텀의 속도 v, Adam의 m, v 등)는 optimizer 객체가 스스로 기억한다
- 가중치 감소(weight decay)도 인자 하나로: `weight_decay=1e-4`

6

torch.utils.data: 데이터와 미니배치

Dataset과 DataLoader

```
X = torch.randn(10, 784)
y = torch.arange(10) % 3
ds = TensorDataset(X, y)
print(len(ds))
x0, y0 = ds[0]
print(x0.shape, y0)
train_ds, val_ds = random_split(ds, [8, 2])
print(len(train_ds), len(val_ds))
loader = DataLoader(train_ds, batch_size=3,
                    shuffle=True)
for xb, yb in loader:
    print(tuple(xb.shape), yb)
```

```
10
torch.Size([784]) tensor(0)
8 2
(3, 784) tensor([2, 2, 1])
(3, 784) tensor([2, 0, 0])
(2, 784) tensor([0, 1])
```

- 마지막 미니배치는 남은 **2**개
- `shuffle=True`: epoch마다 순서를 새로 섞는다

- `TensorDataset(X, y)`: 입력과 정답을 묶은 데이터셋. `ds[i]`는 i번째 `(x, y)` 한 쌍
- `random_split`: 데이터셋을 무작위로 나눈다 (훈련 / 검증)
- `DataLoader`: 데이터셋에서 미니배치를 하나씩 꺼내 주는 **반복자**

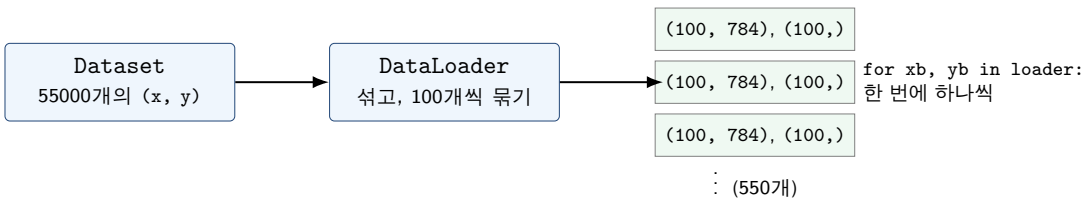
DataLoader가 대신하는 코드

NumPy (우리가 짠 코드)

```
for epoch in range(epochs):
    perm = np.random.permutation(N)
    for i in range(0, N, batch_size):
        idx = perm[i:i + batch_size]
        xb, tb = x_train[idx], t_train[idx]
        ...
```

PyTorch

```
loader = DataLoader(train_set,
                    batch_size=100,
                    shuffle=True)
for epoch in range(epochs):
    for xb, yb in loader:
        ...
```

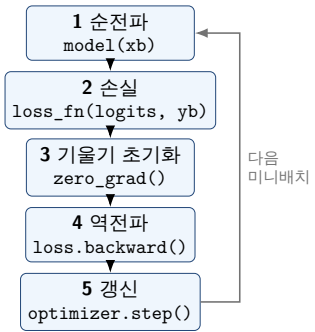


7

학습 루프의 5줄

모든 PyTorch 학습 코드의 뼈대

```
for xb, yb in train_loader:
    logits = model(xb)          # 1
    loss = loss_fn(logits, yb)  # 2
    optimizer.zero_grad()       # 3
    loss.backward()             # 4
    optimizer.step()            # 5
```



- 순전파가 그래프를 기록하고
- 역전파가 모든 `.grad`를 채우고
- `step()`이 그 `.grad`로 갱신한다

모델이 MLP든 CNN이든, 데이터가 MNIST든 무엇이든 **이 5줄은 그대로**이다. 바뀌는 것은 `model`, `loss_fn`, `optimizer`, 데이터뿐.

NumPy 코드와 한 줄씩 비교

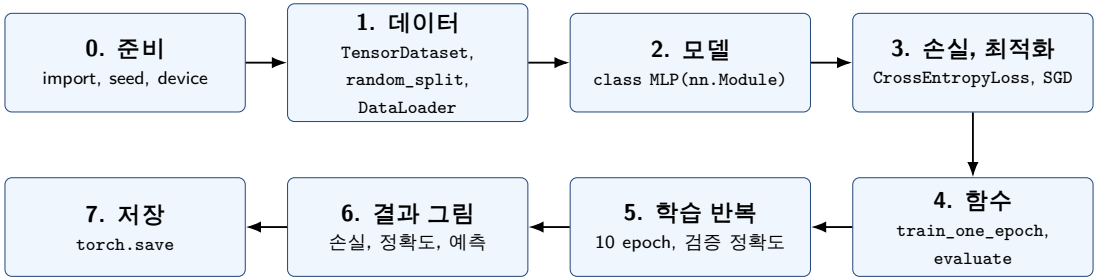
단계	NumPy (우리 코드)	PyTorch
순전파 + 손실	`loss = network.loss(X_batch, T_batch)`	`loss = loss_fn(model(xb), yb)`
기울기 초기화	(필요 없음: 매번 새로 계산해 덮어씀)	`optimizer.zero_grad()`
역전파	`grads = network.backward()` (계층마다 직접 쓴 backward)	`loss.backward()`
갱신	`params[key] -= lr * grads[key]`	`optimizer.step()`

- 구조는 **똑같다**. 우리가 지금까지 배운 것이 그대로 PyTorch 안에 들어 있다
- 그래서 원리를 아는 사람은 PyTorch 코드를 **읽을 수** 있고, 오류가 났을 때 **고칠 수** 있다

8

mnist_torch.py 코드 읽기

코드의 전체 구성



- 문제: MNIST 28×28 손글씨 숫자 $\rightarrow$ 0–9 분류 (다중분류 시간과 같은 문제)
- 모델: 은닉층 하나(100개)의 완전연결 신경망 $784 \rightarrow 100 \rightarrow 10$, ReLU, SGD (학습률 0.1)
- 실행: 파일이 있는 폴더에서 `python mnist_torch.py`

0. 준비

```
import os
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader, random_split
from sklearn.datasets import fetch_openml

torch.manual_seed(0)
os.makedirs("figures", exist_ok=True)

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device}")
```

- `from torch import nn`: 이제 `torch.nn.Linear` 대신 `nn.Linear`로 쓸 수 있다
- `torch.manual_seed(0)`: 가중치 초기화, `random_split`, `shuffle`의 난수를 고정 → 실행할 때마다 같은 결과
- `device`: GPU가 있으면 `"cuda"`, 없으면 `"cpu"` (이 컴퓨터에서는 `Using cpu`)

1. 데이터 (1): 불러와서 텐서로

```
X, y = fetch_openml("mnist_784", version=1, as_frame=False, return_X_y=True)
X = torch.tensor(X / 255.0, dtype=torch.float32)   # (70000, 784)
y = torch.tensor(y.astype(np.int64))               # (70000,)
```

- `fetch_openml`: MNIST 70000장을 내려받는다 (처음 한 번만, 이후에는 컴퓨터에 저장된 것을 사용)
  - `X`: NumPy 배열 (70000, 784), 픽셀 값 0–255. 이미 한 줄로 **펼쳐져** 있다
  - `y`: 정답. 문자열 `'5'`, `'0'`, … 로 들어 있다
- `X / 255.0`: 0–1로 정규화 (다중분류 시간과 같다)
- `dtype=torch.float32`: NumPy의 `float64`를 가중치와 같은 `float32`로
- `y.astype(np.int64)`: 문자열 → 정수. `CrossEntropyLoss`가 요구하는 `int64`
- **one-hot으로 바꾸지 않는다**: 클래스 번호 그대로 쓴다

1. 데이터 (2): 데이터셋과 DataLoader

```
train_val = TensorDataset(X[:60000], y[:60000])
test_set  = TensorDataset(X[60000:], y[60000:])
train_set, val_set = random_split(train_val, [55000, 5000])

batch_size = 100
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_set, batch_size=1000)
test_loader  = DataLoader(test_set, batch_size=1000)
```

데이터	개수	용도	DataLoader
훈련 (train)	55000	매개변수 학습	100개씩, 섞음 (550개 묶음)
검증 (validation)	5000	epoch마다 정확도 확인, 설정 비교	1000개씩, 섞지 않음
테스트 (test)	10000	**마지막에 한 번만** 최종 성능	1000개씩, 섞지 않음

- 평가에는 기울기가 필요 없어 메모리를 덜 쓰므로 미니배치를 크게(1000) 해도 된다. 순서도 상관없다

2. 모델: class MLP(nn.Module)

```
class MLP(nn.Module):
    def __init__(self, in_features=784,
                 hidden=(100,), num_classes=10):
        super().__init__()
        layers = []
        for h in hidden:
            layers += [nn.Linear(in_features, h),
                       nn.ReLU()]
            in_features = h
        layers.append(nn.Linear(in_features,
                                num_classes))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)          # logits
```

- nn.Module을 상속
- super().__init__(): 부모 클래스 초기화. **반드시 첫 줄에**
- hidden: 은닉층 크기의 튜플. (100,): 1개, (256, 128): 2개
- 반복문으로 Linear–ReLU를 쌓고, 마지막에 출력층 Linear
- *layers: 리스트를 풀어서 인자로 전달
- forward: 점수(logits)를 돌려준다. **softmax 없음**
- backward: **없음**

2. 모델: 만들고 확인하기

```
model = MLP(hidden=(100,)).to(device)
print(model)
print("number of parameters:", sum(p.numel() for p in model.parameters()))
```

```
MLP(
  (net): Sequential(
    (0): Linear(in_features=784, out_features=100, bias=True)
    (1): ReLU()
    (2): Linear(in_features=100, out_features=10, bias=True)
  )
)
number of parameters: 79510
```

- `print(model)`: 모델의 구조를 보여 준다. 처음에 항상 확인하는 습관을 들이자
- `.to(device)`: 모든 매개변수를 `device`로
- `model(xb)`라고 호출하면 `forward(xb)`가 실행된다

3. 손실 함수와 최적화 기법

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

- `loss_fn(logits, yb)`: softmax + cross entropy, 미니배치 평균
- `optimizer`: `model.parameters()`로 넘겨받은 4개의 텐서(가중치 2, 편향 2)를 갱신한다
- 다른 기법으로 바꾸려면 이 한 줄만 바꾸면 된다:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

코드 구조에 대한 메모

아래의 `train_one_epoch`는 `loss_fn`과 `optimizer`를 인자로 받지 않고 바깥(전역) 변수를 그대로 쓴다. 짧은 예제라서 이렇게 했다. 큰 프로그램에서는 인자로 넘기는 것이 좋다.

4. 한 epoch 학습: `train_one_epoch`

```
def train_one_epoch(model, loader):
    model.train()
    losses = []
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)

        logits = model(xb)
        loss = loss_fn(logits, yb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        losses.append(loss.item())
    return losses
```

- `model.train()`: 학습 모드로 (앞 epoch의 `evaluate`가 평가 모드로 바꿔 두었다)
- `for xb, yb in loader`: 섞인 미니배치 550개를 차례로
  - `xb`: `(100, 784)`, `yb`: `(100,)`
- `.to(device)`: 모델과 같은 곳으로
- 가운데 5줄: **학습 루프의 뼈대** 그대로
- `loss.item()`: 텐서 → Python 숫자로 기록 (그래프를 붙잡고 있지 않도록)
- 미니배치마다의 손실 550개를 돌려준다

4. 평가: evaluate

```python
@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    correct, total = 0, 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        pred = model(xb).argmax(dim=1)
        correct += (pred == yb).sum().item()
        total += yb.numel()
    return correct / total
```

- `@torch.no_grad()`: 함수 안에서 그래프를 기록하지 않는다 (데코레이터: 다음 쪽)
- `model.eval()`: 평가 모드
- `argmax(dim=1)`: 점수가 가장 큰 클래스 (softmax를 하지 않아도 순서는 같다)
- `pred == yb`: True/False 텐서. `.sum()`으로 맞힌 개수
- `yb.numel()`: 이 미니배치의 데이터 수
- 정확도 = 맞힌 수 / 전체 수

부연: 데코레이터(decorator)란?

- def 바로 위의 @이름: 함수를 **다른 함수에 넣어 감싼** 새 함수로 바꿔치기하는 Python 문법

```
def shout(func):                # takes a function,
    def wrapper(*args):         # returns a new one
        print("  calling", func.__name__)
        return func(*args)
    return wrapper

@shout                          # square = shout(square)
def square(a):
    return a * a

print("square(3) =", square(3))
```

```
  calling square
square(3) = 9
```

- @shout는 아래 한 줄과 **똑같다**: square = shout(square)
- 원래 함수의 앞뒤에 일을 덧붙인다 (여기서는 출력 한 줄)
- 함수 본문은 **고치지 않는다**

부연: @torch.no_grad()

데코레이터로

```
@torch.no_grad()
def predict(model, x):
    return model(x)

print(predict(lin, torch.ones(1, 2)).
     requires_grad)
```

```
False
```

같은 뜻: with 문으로

```
def predict(model, x):
    with torch.no_grad():
        return model(x)
```

- `@torch.no_grad()`: 함수 **전체**를 `with torch.no_grad():` 안에서 실행한다
- 괄호 `()`가 붙는다: `torch.no_grad()`로 **만든 객체**가 데코레이터 역할을 한다
- `evaluate`처럼 함수 하나가 통째로 평가용일 때 편하다. 몇 줄만 감쌀 때는 `with` 문
- 그 밖에 자주 보는 데코레이터: `@staticmethod`, `@property` (클래스 안에서)

5. 학습 반복과 실행 결과

```
epochs = 10
loss_history, val_acc_history = [], []
for epoch in range(1, epochs + 1):
    loss_history += train_one_epoch(
                        model, train_loader)
    val_acc = evaluate(model, val_loader)
    val_acc_history.append(val_acc)
    print(f"epoch {epoch:2d} | loss "
          f"{loss_history[-1]:.4f} | "
          f"val acc {val_acc:.4f}")

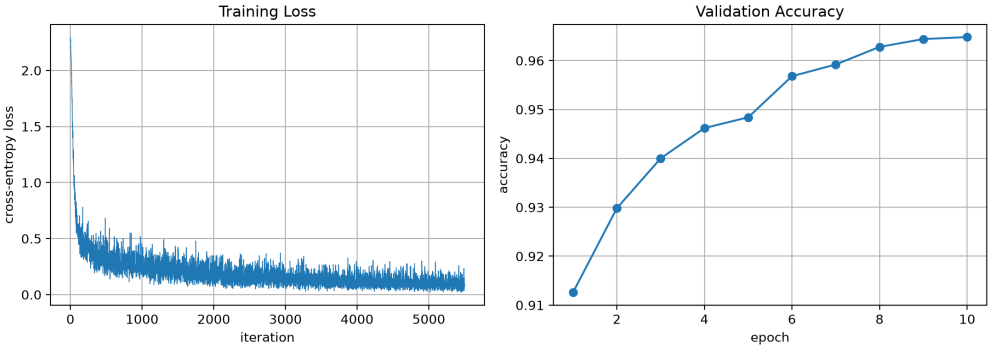
test_acc = evaluate(model, test_loader)
print(f"test accuracy: {test_acc:.4f}")
```

```
epoch  1 | loss 0.3039 | val acc 0.9126
epoch  2 | loss 0.1827 | val acc 0.9298
epoch  3 | loss 0.1847 | val acc 0.9400
epoch  4 | loss 0.1535 | val acc 0.9462
epoch  5 | loss 0.1379 | val acc 0.9484
epoch  6 | loss 0.1828 | val acc 0.9568
epoch  7 | loss 0.1299 | val acc 0.9592
epoch  8 | loss 0.1276 | val acc 0.9628
epoch  9 | loss 0.1147 | val acc 0.9644
epoch 10 | loss 0.0701 | val acc 0.9648
test accuracy: 0.9694
```

생각해 보기

test 정확도 **96.94%**. 출력된 loss는 그 epoch의 **마지막 미니배치 하나**의 손실이다 (`loss_history[-1]`). 그래서 epoch 6처럼 오르락내리락한다. epoch 평균을 출력하려면 어떻게 고치면 될까?

6. 결과 그림 (1): 손실과 검증 정확도



왼쪽: 미니배치마다의 손실 ($550 \times 10 = 5500$ iteration). 오른쪽: epoch마다의 검증 정확도

- 손실은 처음에 빠르게 줄고, 미니배치마다 데이터가 달라서 **들쭉날쭉**하다 (확률적 경사하강법)
- 검증 정확도는 아직 오르고 있다 → epoch를 늘리면 더 좋아질 수 있다

6. 결과 그림 (2): 예측 확인

```
model.eval()
with torch.no_grad():
    xb, yb = next(iter(test_loader))                    # first test mini-batch
    probs = torch.softmax(model(xb.to(device)), dim=1).cpu()
pred = probs.argmax(dim=1)
...
    a.imshow(xb[k].reshape(28, 28), cmap="gray")        # 784 -> 28 x 28
    a.set_title(f"pred {pred[k].item()}\n({probs[k, pred[k]]:.2f})", fontsize=8)
```



- `next(iter(test_loader))`: DataLoader에서 첫 미니배치 하나만 꺼낸다
- 확률이 필요하므로 여기서는 `torch.softmax`를 직접 쓴다. 그림을 그리려고 `.cpu()`로 가져온다

7. 학습한 모델 저장하고 다시 불러오기

저장 (mnist_torch.py의 마지막 줄)

```
torch.save(model.state_dict(), "mnist_mlp.pt")
```

불러오기 (다른 프로그램에서)

```
model2 = MLP(hidden=(100,))                          # the same structure first
model2.load_state_dict(torch.load("mnist_mlp.pt"))   # then the trained numbers
model2.eval()
```

- state_dict: {"net.0.weight": ..., "net.0.bias": ..., "net.2.weight": ..., "net.2.bias": ...}
- 저장되는 것은 **숫자(매개변수)뿐**이다. 구조는 코드(class MLP)로 다시 만든다
- 학습에 몇 시간, 며칠이 걸리는 모델도 한 번 저장해 두면 바로 예측에 쓸 수 있다
- 공개된 학습된 모델을 내려받아 쓰는 것(전이학습)도 이 방법이다

NumPy 버전과 무엇이 달라졌나

	NumPy (`mnist_lab.py`)	PyTorch (`mnist_torch.py`)
정답 표현	one-hot `(N, 10)`	클래스 번호 `(N,)`, `int64`
계층	`Affine`, `Relu` 직접 작성	`nn.Linear`, `nn.ReLU`
가중치 초기화	직접 (`0.01 * randn` 등)	자동
출력층	`SoftmaxWithCrossEntropy`	`nn.CrossEntropyLoss` (logits 입력)
역전파	계층마다 `backward`, 기울기 검사	`loss.backward()`
갱신	`W -= lr * dW`	`optimizer.step()`
미니배치	`permutation`으로 섞고 자르기	`DataLoader(shuffle=True)`
GPU	불가능	`.to(device)`

- 코드는 짧아졌지만 **하는 일은 같다**. 각 줄 뒤에서 무엇이 계산되는지 우리는 이미 알고 있다

9

정리와 실습

정리

이번 시간의 핵심

- **텐서**: NumPy 배열 + GPU. 속성 `shape`, `dtype` (`float32`, 레이블은 `int64`), `device`
- **자동 미분**: 순전파가 그래프를 기록하고 `loss.backward()`가 모든 `.grad`를 채운다. 기울기는 더해지므로 매번 `zero_grad()`
- **`nn`**: `nn.Module`을 상속하고 `forward`만 쓴다. `nn.Linear`의 가중치는 `(out, in)`
- **`nn.CrossEntropyLoss`**: logits와 클래스 번호를 받는다. 모델에 softmax를 넣지 않는다
- **`optim`**: `zero_grad()`, `step()`
- **`utils.data`**: `TensorDataset`, `random_split`, `DataLoader`
- **학습 루프 5줄**: 순전파 → 손실 → `zero_grad` → `backward` → `step`
- 평가할 때: `model.eval()`, `torch.no_grad()`

같은 틀로 CNN도: 바꾸는 것은 모델뿐

```
model = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),  # (N,1,28,28) -> (N,16,28,28)
    nn.ReLU(),
    nn.MaxPool2d(2),                             # -> (N,16,14,14)
    nn.Flatten(),                                # -> (N,3136)
    nn.Linear(16 * 14 * 14, 10),                 # -> (N,10)
).to(device)
```

- CNN 시간에 배운 계층들이 그대로 있다. 매개변수 31530개
- 입력은 펼치지 않고 이미지 모양 그대로: `xb.reshape(-1, 1, 28, 28)`
- 손실 함수, 최적화 기법, `train_one_epoch`, `evaluate`는 **한 글자도 바꾸지 않는다**

실습 과제

`mnist_torch.py`를 복사해서 하나씩 바꿔 보고, 검증 정확도와 test 정확도를 기록하자.

1. **epoch 평균 손실**을 출력하도록 고치기 (힌트: `np.mean`)
2. 은닉층을 `hidden=(256, 128)`로 바꾸기. 매개변수 수는 몇 개인가? 손으로 먼저 계산해 보자
3. 최적화 기법을 `torch.optim.Adam(model.parameters(), lr=0.001)`로 바꾸기
4. `MLP` 클래스에 `nn.Dropout(0.2)`를 ReLU 뒤에 넣기. `model.train()`과 `model.eval()`의 역할을 설명하기
5. `optimizer.zero_grad()`를 **지우고** 실행해 보기. 무슨 일이 생기고, 왜 그런가?
6. test 데이터 중 **틀린 이미지**를 찾아 예측과 정답을 함께 그리기
7. 앞 슬라이드의 CNN으로 모델을 바꾸기 (입력 `reshape`에 주의). MLP와 정확도 비교
8. 저장한 `mnist_mlp.pt`를 새 프로그램에서 불러와 test 정확도가 같은지 확인하기